

Model Understanding and Data Exploration Notes

By Jay Bell

July 16, 2026

1 How My DAG Model Worked (Plain Language for a Medical Audience)

I built a simple directed acyclic graph (DAG), which was a map of assumed cause-and-effect relationships. Each dot represented one variable:

- The first variable was **Obesity**.
- The second variable was **Diabetes**.
- The third variable was **Cancer**.

I drew one-way arrows between these variables to represent the direction of influence. Because the graph was acyclic, arrows did not loop back.

I used these arrows:

- **Obesity** → **Diabetes**
- **Obesity** → **Cancer**
- **Diabetes** → **Cancer**

I used this structure to ask a focused question: “If diabetes changed, how much did cancer risk change, after accounting for the pathway through obesity?”

I also tested whether the result was stable using internal checks. For example, I repeated the analysis under perturbations (such as placebo-style and subset checks) to see whether the effect estimate remained directionally similar.

So, the DAG served as an explicit assumption map. It helped me separate correlation from a more defensible causal interpretation, and it made the model logic transparent for review.

2 Merged NHANES Columns and How I Explored the Data

The merged file was:

`api/nhanes_data/nhanes_merged.csv`

The merge script kept original NHANES variable names and added a prefix so I could track each source table. All tables were joined using `SEQN`, which was the participant ID key.

2.1 What the Prefixes Meant

I organized the merged columns into four main families:

- **DEMO_...**: Demographics (age, sex, race/ethnicity, income, survey design variables).
- **BMX_...**: Body measurements (height, weight, BMI, waist, and related body metrics).
- **DIQ_...**: Diabetes questionnaire responses.
- **MCQ_...**: Medical conditions questionnaire responses (including cancer-related questions).

Examples from the merged header:

- `DEMO_RIDAGEYR`: age in years.
- `BMX_BMXBMI`: body mass index.
- `DIQ_DIQ010`: diabetes status question.
- `MCQ_MCQ220`: cancer status question used to derive the model target.

2.2 Model-Ready Columns in the Same File

The preparation step also created three model-ready columns:

- **Obesity**: derived from `BMX_BMXBMI` ($\text{BMI} \geq 30$ mapped to 1).
- **Diabetes**: derived from `DIQ_DIQ010` (Yes/No recoded to 1/0).
- **Cancer**: derived from `MCQ_MCQ220` (Yes/No recoded to 1/0).

So, the merged dataset contained both raw NHANES-prefixed variables and these binary fields used directly by the DAG model.

2.3 How I Explored the Data

I explored the file in a practical sequence:

1. I checked the header to see all column names.
2. I looked at a few rows to confirm values looked reasonable.
3. I checked missing values to find sparse columns.
4. I summarized key variables to understand distributions.
5. I counted class balance for **Obesity**, **Diabetes**, and **Cancer**.

2.4 Reproducible Exploration Commands

Python (pandas) quick scan:

```
import pandas as pd

df = pd.read_csv("api/nhanes_data/nhanes_merged.csv")

# 1) Shape and first columns
print(df.shape)
print(df.columns[:30].tolist())

# 2) Preview rows
print(df.head(5))

# 3) Missingness (top 20 most missing columns)
missing = df.isna().mean().sort_values(ascending=False)
print(missing.head(20))

# 4) Summary stats for numeric columns
print(df.describe(include=["number"]).T.head(20))

# 5) Class balance for model columns
for col in ["Obesity", "Diabetes", "Cancer"]:
    if col in df.columns:
        print("\n", col)
        print(df[col].value_counts(dropna=False))
```

Header-only check from terminal:

```
python - <<'PY'
import pandas as pd
df = pd.read_csv("api/nhanes_data/nhanes_merged.csv", nrows=0)
print(len(df.columns))
print(df.columns.tolist()[:40])
PY
```

These steps helped me quickly identify what each column family represented, which variables had acceptable completeness, and how DAG inputs were generated from the merged NHANES dataset.